

This worksheet is designed to pair with your **Interactive 2's Complement Visualizer**.

Since the tool specifically visualizes **Signed (2's Complement)** data, the worksheet scaffolds the learning by asking students to contrast what they *see* (Signed) with what they *calculate* (Unsigned/Hex) based on the same binary patterns.

Worksheet: Exploring the Circle of Bits

Objective: Investigate how computers store numbers, how they "wrap around" (overflow), and how the same pattern of zeros and ones can mean different things.

Part 1: The "Unsigned" Reality vs. The "Signed" Visual

Set the visualizer to **4-Bit System** and the mode to **Intro**.

1. **Pattern Spotting:** Click the **+1** button to count up from 0. Watch the binary bits.
 - What binary pattern represents the number 7? _____
 - Click **+1** again. The tool shows **-8**.
 - **Stop!** If we were treating these 4 bits as a standard **Unsigned Integer** (just normal binary counting), what *should* come after 7 (\$0111_2\$)?
 - Binary: _____
 - Decimal: _____
2. **Hexadecimal Translation:** Find the following values on the wheel. Write down the 4-bit Binary pattern you see, then translate that pattern into a single **Hexadecimal** digit.

Tool Value (Signed)	Binary Pattern (Visible)	Hexadecimal Digit (Calculated)
0	0000	0
-1	1111	F
-2	_____	_____
5	_____	_____
-8	_____	_____

Critical Thinking: Why is Hexadecimal useful for representing these 4-bit patterns, regardless of whether the number is positive or negative?

Part 2: Cracking the Code (2's Complement)

Switch to the **Sign** tab.

3. **The MSB Rule:** Toggle the bits in the "Explorative Lab" (or click different numbers on the wheel).
 - What is the **first bit** (Most Significant Bit) for every positive number? _____

- What is the **first bit** for every negative number? ____

4. **The Conversion Challenge:** You want to find the binary for **-3**.

- **Step A:** Write positive 3 in binary (4-bit): ____
- **Step B:** Flip all the bits (0→1, 1→0): ____
- **Step C:** Add 1 to the result of Step B: ____
- **Step D:** ☐ Yes [] No

Part 3: The Limits of Storage (Range)

Switch to the **Range** tab.

5. **Observation:**

- What is the maximum positive number in a 4-bit system? ____
- What is the most negative number in a 4-bit system? ____

6. **Prediction (High Order Thinking):** Change the config to a **5-Bit System**. Do not look at the values yet.

- Calculate the new Maximum Positive value ($2^{n-1} - 1$): ____
- Calculate the new Most Negative value (-2^{n-1}): ____
- *Now check the tool to verify.*

Part 4: Overflow & The "Wheel"

Switch to the **Add** tab. Ensure you are back on **4-bits**.

7. **Visualizing Math:** Set the calculation to: **5 + 2**.

- Result: 7
- ☐ Yes [] No

8. **Forcing the Error (Overflow):** Set the calculation to: **5 + 4**.

- Mathematically, $5 + 4 = 9$.
- What result does the tool give? ____
- **Why?** Explain using the "Wheel" visualization. Why did the answer "wrap around" to a negative number?

9. **Unsigned vs Signed Overflow:** Look at the binary addition for the problem above ($0101 + 0100 = 1001$).

- In **Signed** arithmetic (the tool), 1001 is -7 (Overflow occurred).
- In **Unsigned** arithmetic, 1001 is _____. Did overflow occur if we treat these as unsigned numbers? (Hint: Can a 4-bit unsigned system hold the number 9?)

Part 5: Master Challenge (Assessment)

Scenario: You are programming a car's odometer (mileage counter) using a 4-bit variable.

1. **The Crash:** The odometer reads 0111 (\$7\$ miles). You drive 1 more mile.
 - If the system is **Signed**, what does the odometer read now? _____
 - If the system is **Unsigned**, what does the odometer read now? _____
2. **The Bug Fix:** Your boss wants the car to track up to 20 miles.
 - Can you do this with 4 bits? Explain why or why not.
 - What is the minimum number of bits you need? _____
3. **Hex Translation:** If your odometer reads 1101:
 - What is the decimal value if Signed? _____
 - What is the decimal value if Unsigned? _____
 - What is the Hexadecimal representation? _____